Software Architecture Document (SAD)

Redbox Apparel — Multi-Tenant Retail & E-Commerce Platform

Field	Value
Version	1.0
Date	2026-03-08
Status	Living Document
Stack	Next.js 15 · React 19 · Convex · Clerk · TailwindCSS

Table of Contents

1. Introduction
2. Architectural Goals & Constraints
3. System Overview
4. Architectural Views
5. Data Architecture
6. Security Architecture
7. Integration Architecture
8. Deployment Architecture
9. Cross-Cutting Concerns
10. Technology Stack
11. Appendix

---

1. Introduction

1.1 Purpose

This document describes the software architecture of **Redbox Apparel**, a full-stack retail management and e-commerce platform. It serves as the single source of truth for developers, stakeholders, and future maintainers to understand system structure, key design decisions, and quality-attribute trade-offs.

1.2 Scope

Redbox Apparel covers:

- **Customer-facing storefront** — product browsing, cart, checkout, reservations, wishlists, loyalty
- **Point-of-Sale (POS)** — in-store transactions, barcode scanning, receipts, reconciliation
- **Inventory management** — multi-branch stock tracking, low-stock alerts, aging tiers, batch tracking
- **Inter-branch transfers** — request/pack/ship/receive workflow with driver delivery
- **Promotions engine** — percentage, fixed amount, buy-X-get-Y, tiered discounts with complex scoping
- **Analytics & reporting** — HQ/branch dashboards, BIR tax reports, demand intelligence, product movers
- **Supplier portal** — external supplier access for product proposals
- **AI features** — restock suggestions, branch performance scoring

1.3 Definitions

Term	Definition
Style	A product design (e.g., "Urban Crew Neck Tee"). Has a base price.
Variant	A purchasable SKU — a Style × Size × Color × Gender combination

Branch	A physical location, either `retail` (store) or `warehouse` (HQ/central)
Demand Log	A customer request for an out-of-stock or unavailable product
Aging Tier	Inventory freshness classification: green (<90 days), yellow (90-180), red (>180)

---

2. Architectural Goals & Constraints

2.1 Quality Attributes

Priority	Attribute	Target
1	**Real-time reactivity**	All data changes propagate to connected clients instantly via Convex subscriptions
2	**Role-based security**	8 roles with strict route + function-level access control
3	**Offline resilience**	POS continues operating during network outages via offline queue
4	**Multi-branch isolation**	Branch users see only their own data; HQ sees everything
5	**Auditability**	All state-changing operations logged to append-only audit trail

2.2 Constraints

- **Backend**: Convex — all server logic runs as Convex functions (queries, mutations, actions). No custom Express/API server.
- **Authentication**: Clerk — OAuth provider with custom JWT claims for role propagation.
- **Hosting**: Vercel (Next.js frontend) + Convex Cloud (backend).
- **Database**: Convex document store — no SQL, no joins. All relationships resolved in application code.
- **Real-time**: Convex subscriptions — no WebSocket management needed, built into the platform.

2.3 Key Design Decisions

Decision	Rationale
Convex over traditional REST API + database	Real-time subscriptions, automatic caching, type-safe end-to-end, zero infrastructure management
Clerk for auth	Managed OAuth, pre-built UI components, webhook-based user sync, custom JWT claims for RBAC
Next.js App Router	Server/client component model, route groups for role isolation, middleware for edge auth
Cursor-based pagination everywhere	Consistent UX, efficient for Convex's document model, avoids offset-based issues at scale
Single admin route (merged Admin + HQ)	Reduced code duplication — HQ and Admin share most features, differentiated by role checks

---

3. System Overview

3.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                        CLIENTS                               │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────────┐  │
│  │ Customer │  │   POS    │  │  Admin   │  │ Branch/WH/   │  │
```

```
│  │Storefront │  │ Terminal │  │  Panel   │  │ Driver/Suppl │   │
│  └──────┬────┘  └────┬─────┘  └────┬─────┘  └──────┬───────┘   │
└─────────┼────────────┼────────────┼───────────────┼───────────┘
          │            │            │               │
     ┌────▼────────────▼────────────▼───────────────▼────┐
     │             NEXT.JS 15 (VERCEL)                   │
     │  ┌──────────┐  ┌──────────┐  ┌──────────────┐    │
     │  │ Middleware │  │  App Router│  │  Components    │  │
     │  │ (Edge Auth)│  │ (6 groups) │  │  (Shared/UI)   │  │
     │  └─────┬────┘  └─────┬────┘  └──────────────┘    │
     └────────┼────────────┼──────────────────────────┘
              │            │
     ┌────────▼────────────▼──────────────────────────┐
     │               CONVEX CLOUD                     │
     │  ┌─────────┐  ┌─────────┐  ┌──────────────┐    │
     │  │ Queries   │  │Mutations │  │    Actions       │   │
     │  │(read-only)│  │(writes)  │  │(side-effects)    │   │
     │  └─────────┘  └─────────┘  └──────────────┘    │
     │  ┌─────────┐  ┌─────────┐  ┌──────────────┐    │
     │  │ Database  │  │ Storage  │  │  Cron Jobs       │   │
     │  │(43 tables)│  │ (files)  │  │  (hourly/daily)  │   │
     │  └─────────┘  └─────────┘  └──────────────┘    │
     └────────────────────────────────────────────────┘
              │
     ┌────────▼──────────────────────────────┐
     │          EXTERNAL SERVICES            │
     │  ┌──────┐  ┌──────┐  ┌────────────┐   │
     │  │ Clerk  │  │ Resend │  │   Svix       │  │
     │  │ (Auth) │  │ (Email)│  │ (Webhooks)   │  │
     │  └──────┘  └──────┘  └────────────┘   │
     └────────────────────────────────────────┘
```

3.2 Route Groups & User Portals

Route Group	Users	Key Features
`(customer)`	Public / Registered customers	Browse, cart, checkout, reservations, wishlists, orders, account
`/admin`	Admin, HQ Staff	Full catalog CRUD, promotions, banners, analytics, settings, seed
`/branch`	Branch Managers, Viewers	Branch dashboard, stock levels, transfers, alerts, demand
`/pos`	Cashiers, Managers, Admin	Transaction processing, barcode scan, receipts, reconciliation
`/warehouse`	Warehouse/HQ Staff	Transfer fulfillment, receiving, logistics, restock AI
`/driver`	Drivers	Delivery assignments and confirmation
`/supplier`	Suppliers	Product proposals and catalog access

---

4. Architectural Views

4.1 Component View — Backend Modules

```
convex/
├── _helpers/              # Cross-cutting utilities
│   ├── permissions.ts     # requireAuth(), requireRole()
```

```
│   ├── withBranchScope.ts  # Branch isolation enforcement
│   ├── transferStock.ts    # Stock reservation/release
│   ├── auditLog.ts         # Append-only audit entries
│   ├── promoCalculations.ts# Multi-type discount engine
│   ├── taxCalculations.ts  # VAT & senior/PWD discounts
│   └── internalInvoice.ts  # Transfer invoice generation
│
├── auth/                   # User identity & branch mgmt
├── catalog/                # Brands → Categories → Styles → Variants → Images
├── inventory/              # Stock levels, alerts, batch tracking
├── pos/                    # Transactions, shifts, reconciliation
├── transfers/              # Inter-branch stock movement
├── storefront/             # E-commerce: cart, orders, wishlist
├── dashboards/             # Analytics & reporting (HQ + Branch)
├── demand/                 # Customer demand tracking
├── reservations/           # Product reservations
├── admin/                  # Settings, colors, sizes, promos, banners
├── ai/                     # Restock suggestions, branch scoring
├── logistics/              # Driver assignments & deliveries
├── invoices/               # Internal transfer invoices
├── suppliers/              # Supplier portal
└── audit/                  # Audit log queries
```

4.2 Component View — Frontend

```
components/
├── ui/                     # Shadcn/ui primitives (button, card, dialog, etc.)
├── shared/                 # Cross-portal components
│   ├── TablePagination     # Cursor-based pagination controls
│   ├── ProductCard         # Admin product card
│   ├── BarcodeScanner      # html5-qrcode integration
│   ├── ReceiptPDF          # @react-pdf/renderer receipts
│   ├── ConnectionIndicator # Online/offline status
│   └── ErrorBoundary       # Error boundary wrapper
├── customer/               # Storefront-specific
│   ├── CustomerProductCard # Product card with quick view
│   ├── HeroSection         # Homepage hero carousel
│   ├── AnnouncementBar     # Top announcement strip
│   └── QuickViewSheet      # Product quick preview sheet
├── pos/                    # POS-specific
│   ├── POSCartPanel        # Cart sidebar
│   ├── POSProductGrid      # Product grid with search
│   └── ReconciliationPanel # End-of-day reconciliation
└── providers/              # Context providers
    ├── ConvexClientProvider # Convex + Clerk integration
    ├── BrandProvider       # Brand context
    └── POSCartProvider     # POS cart state
```

4.3 Process View — Key Workflows

4.3.1 POS Transaction Flow

```
Cashier scans barcode
  → POSProductGrid looks up variant by barcode
  → POSCartProvider adds to cart state
  → Cashier applies promo (optional)
```

```
    → promoCalculations.ts validates scope & calculates discount
  → Cashier selects payment method
  → pos/transactions.ts:createTransaction mutation
    → Validates inventory availability
    → Deducts stock (inventory.quantity -= qty)
    → Creates transaction + transactionItems records
    → Calculates VAT (taxCalculations.ts)
    → Logs audit entry
  → Receipt generated (ReceiptPDF component)
```

4.3.2 Inter-Branch Transfer Flow

```
Branch Manager creates transfer request
  → transfers/requests.ts:createTransferRequest
  → Status: "requested"
  → Stock reserved: inventory.reservedQuantity += qty

Warehouse reviews & packs
  → transfers/fulfillment.ts:packTransfer
  → Status: "packed"
  → Internal invoice generated

Driver assigned & picks up
  → logistics/assignments.ts:assignDriver
  → Status: "in_transit"

Driver delivers to branch
  → logistics/deliveries.ts:driverConfirmDelivery
  → OR transfers/fulfillment.ts:confirmTransferDelivery
  → Status: "delivered"
  → Source: clearReservedOnDelivery() removes reserved qty
  → Destination: inventory.quantity += received qty
  → New inventoryBatch created at destination
```

4.3.3 Customer Order Flow

```
Customer browses → adds to cart
  → storefront/cart.ts mutations

Customer checks out
  → storefront/orders.ts:placeOrder
  → Status: "pending" → "confirmed"
  → Stock deducted from nearest retail branch

Warehouse fulfills
  → Status: "processing" → "shipped"
  → Shipment record created (courier tracking)

Customer receives
  → Status: "delivered"
  → Review eligible after delivery
```

4.4 Deployment View

```
┌──────────────────────────┐
│       Vercel Edge        │
│  ┌────────────────────┐  │
│  │   middleware.ts    │  │  ← Route protection, JWT validation
│  └────────────────────┘  │
│  ┌────────────────────┐  │
│  │   Next.js SSR +    │  │  ← App Router, RSC, client components
│  │   Static Pages     │  │
│  └────────────────────┘  │
└──────────────────────────┘
             │ HTTPS
┌────────────▼─────────────┐
│     Convex Cloud         │
│  ┌──────────────────┐    │
│  │ Function Runtime │    │  ← Queries, Mutations, Actions, Crons
│  ├──────────────────┤    │
│  │ Document Store   │    │  ← 43 tables, indexes, real-time sync
│  ├──────────────────┤    │
│  │ File Storage     │    │  ← Product images, logos, banners
│  ├──────────────────┤    │
│  │ Cron Scheduler   │    │  ← Hourly alerts, daily AI scoring
│  └──────────────────┘    │
└──────────────────────────┘
```

---

5. Data Architecture

5.1 Entity Relationship Overview

```
brands (1) ──→ (N) categories (1) ──→ (N) styles (1) ──→ (N) variants
                                          │                    │
                                          │                    ├──→ inventory (per branch)
                                          │                    ├──→ inventoryBatches
                                          │                    ├──→ transactionItems
                                          │                    └──→ cartItems
                                          │
                                          └──→ productImages
                                          └──→ wishlists
                                          └──→ recentlyViewed
                                          └──→ reviews

users ──→ branches (assigned)
      ──→ transactions (created by)
      ──→ cashierShifts
      ──→ auditLogs

customers ──→ customerAddresses
          ──→ carts ──→ cartItems
          ──→ orders ──→ orderItems
          ──→ wishlists
          ──→ reservations
          ──→ loyaltyAccounts ──→ loyaltyTransactions

transfers ──→ transferItems
          ──→ internalInvoices ──→ internalInvoiceItems
```

```
promotions (scoped by: branch, brand, category, style, variant, gender, color, size, aging)
```

5.2 Key Tables Summary

Domain	Tables	Record Estimate
Identity	users, branches, settings	Low volume
Catalog	brands, categories, styles, variants, productImages, colors, sizes	~500-5,000 SKUs
Inventory	inventory, lowStockAlerts, inventoryBatches	Per-branch × per-variant
POS	transactions, transactionItems, cashierShifts, reconciliations	High volume (daily)
Transfers	transfers, transferItems, internalInvoices, internalInvoiceItems	Medium volume
Storefront	customers, addresses, carts, cartItems, orders, orderItems, shipments	Growing with customer base
Engagement	wishlists, reviews, vouchers, voucherRedemptions, loyaltyAccounts, loyaltyTransactions, notifications, recentlyViewed, sizeCharts, banners	Variable
Analytics	demandLogs, demandWeeklySummaries, auditLogs, restockSuggestions, branchScores	Append-heavy
Promotions	promotions	Low volume, complex scoping
Reservations	reservations	Medium volume with expiry

5.3 Indexing Strategy

Convex uses declared indexes for efficient querying. Key indexes include:

- `variants`: `by_style`, `by_barcode`, `by_sku` — fast lookup for POS scanning and catalog browsing
- `inventory`: `by_branch_variant` — unique stock per branch per variant
- `transactions`: `by_branch`, `by_receipt` — branch filtering and receipt lookup
- `productImages`: `by_style` — image retrieval for product display
- `transfers`: `by_source`, `by_destination`, `by_status` — transfer workflow filtering
- `orders`: `by_customer`, `by_status` — customer order history and fulfillment queues

---

6. Security Architecture

6.1 Authentication Flow

```
Browser → Clerk (OAuth) → JWT with custom claims {role, branchId}
  → Next.js middleware (edge) validates route access
  → Convex functions call requireAuth() / requireRole()
  → Branch isolation via withBranchScope()
```

6.2 Role-Based Access Control (RBAC)

Role	Access Level	Portals
`admin`	Full system access	All portals

`manager`	Branch operations + POS	`/branch`, `/pos`
`cashier`	POS only	`/pos`
`hqStaff`	HQ operations	`/admin`, `/warehouse`
`warehouseStaff`	Warehouse operations	`/warehouse`
`viewer`	Read-only branch data	`/branch`
`driver`	Delivery management	`/driver`
`supplier`	Supplier portal	`/supplier`

6.3 Authorization Layers

1. **Edge Middleware** (`middleware.ts`): Route-level protection. Validates JWT, checks role against route group ACL. Lightweight — no crypto verification, relies on Clerk token expiry.
2. **Convex Function Guards** (`_helpers/permissions.ts`):
   - `requireAuth(ctx)` — Verifies active user, detects stale sessions (role mismatch between JWT and DB)
   - `requireRole(ctx, allowedRoles)` — Validates user has one of the specified roles
3. **Branch Isolation** (`_helpers/withBranchScope.ts`):
   - Non-HQ users can only access data from their assigned branch
   - HQ users (admin, hqStaff) bypass branch filtering
   - Enforced at the query/mutation level — not just UI
4. **Audit Trail** (`_helpers/auditLog.ts`):
   - Append-only `auditLogs` table
   - Captures: action, userId, branchId, entityType, entityId, before/after state
   - Cannot be modified or deleted

6.4 Data Protection

- Environment secrets managed via Clerk + Convex dashboard (never committed)
- `encryption.ts` utility for sensitive data at rest
- Webhook verification via Svix for Clerk events
- No direct database access — all reads/writes through Convex functions with auth guards

---

7. Integration Architecture

7.1 External Services

Service	Purpose	Integration Method
Clerk	Authentication & user management	OAuth SDK + webhook sync
Convex	Backend + database + real-time	SDK (React hooks + server functions)
Resend	Transactional email	API calls from Convex actions
Svix	Webhook verification	Clerk webhook endpoint validation

7.2 Webhook Flow (Clerk → Convex)

```
Clerk user event (create/update/delete)
  → POST /api/webhooks/clerk (convex/http.ts)
  → Svix signature verification
  → convex/auth/clerkWebhook.ts processes event
  → Upserts user record in Convex DB
```

7.3 File Storage

Convex Storage handles all file uploads:

- Product images (multiple per style, with primary flag)
- Brand logos and banners
- Homepage banners
- Accessed via `ctx.storage.getUrl(storageId)` — returns CDN URL

---

8. Deployment Architecture

8.1 Environments

Environment	Frontend	Backend
Development	`localhost:3000`	Convex dev instance
Production	Vercel (auto-deploy from main)	Convex Cloud production

8.2 Build & Deploy

```
git push main
  → Vercel builds Next.js (static + SSR)
  → npx convex deploy (schema + functions)
  → Live in ~60 seconds
```

8.3 PWA Support

- `manifest.ts` — app manifest with icons
- `public/sw.js` — service worker for offline caching
- `lib/serviceWorker.ts` — registration helper
- `lib/offlineQueue.ts` — queues POS transactions during network outages

---

9. Cross-Cutting Concerns

9.1 Error Handling

- **Frontend**: ErrorBoundary component wraps route groups; Sonner toasts for user-facing errors
- **Backend**: Convex functions throw typed errors; client-side error handling via try/catch on mutations
- **POS Offline**: Transactions queued locally, synced on reconnection

9.2 Logging & Observability

- **Audit Logs**: All mutations log to `auditLogs` table with before/after state diffs
- **Convex Dashboard**: Built-in function logs, error tracking, real-time data browser
- **Connection Indicator**: Visual online/offline status for POS users

9.3 Pagination

All tables use **cursor-based pagination** (project convention — no exceptions):

- Timestamp-based cursor with over-fetch (`limit + 1`) to detect `hasMore`
- `usePagination` hook in `lib/hooks/usePagination.ts`
- `TablePagination` component in `components/shared/`

9.4 Tax Calculations (Philippine BIR Compliance)

- **VAT**: 12% inclusive — `vatAmount = price - round(price / 1.12)`
- **Senior/PWD Discount**: VAT-exempt base = `round(price / 1.12)` , then 20% discount
- **BIR Reports**: Dedicated `dashboards/birReports.ts` for compliance reporting

9.5 Promotion Engine

Multi-type discount system with complex scoping:

Type	Description
`percentage`	X% off with optional max cap
`fixedAmount`	Flat amount off
`buyXGetY`	Buy X items, get Y free
`tiered`	Spend thresholds unlock discounts

Scoping dimensions: branch, classification, brand, category, style, variant, gender, color, size, inventory aging tier.

9.6 Background Jobs (Crons)

Schedule	Job	Purpose
Hourly	Low-stock sweep	Check inventory against thresholds, create alerts
Hourly	Reservation expiry	Cancel expired reservations, release stock
Daily 5AM	Restock suggestions	AI-generated restock recommendations
Daily 6AM	Branch scoring	Performance metrics across branches
Weekly Monday	Demand summaries	Aggregate weekly demand logs by brand

---

10. Technology Stack

10.1 Core Stack

Layer	Technology	Version
Frontend Framework	Next.js	15
UI Library	React	19
Backend	Convex	1.x
Authentication	Clerk	6.x
Styling	TailwindCSS	3.4
Component Library	Shadcn/ui + Radix UI	Latest
Language	TypeScript	5.x

10.2 Key Libraries

Library	Purpose
`recharts`	Dashboard charts and analytics visualizations
`@react-pdf/renderer`	Receipt and invoice PDF generation
`html5-qrcode`	Barcode/QR code scanning for POS
`papaparse`	CSV parsing for bulk import
`lucide-react`	Icon library
`sonner`	Toast notifications
`class-variance-authority`	Component variant management
`resend`	Transactional email delivery
`svix`	Webhook signature verification

10.3 Development Tools

Tool	Purpose
ESLint	Code linting
TypeScript strict mode	Type safety
Convex CLI	Backend development and deployment
Vercel CLI	Frontend deployment

---

11. Appendix

11.1 File Count Summary

Area	Files	Lines (approx)
Convex backend	50+	~18,000
App pages	82	~15,000
Components	35	~5,000
Lib/utils	11	~1,500
Config	8	~300
Total	**186+**	**~40,000**

11.2 Database Tables (43 total)

Identity & Access: users, branches, settings **Catalog**: brands, categories, styles, variants, productImages, colors, sizes **Inventory**: inventory, lowStockAlerts, inventoryBatches **POS**: transactions, transactionItems, cashierShifts, reconciliations **Transfers**: transfers, transferItems, internalInvoices, internalInvoiceItems **Demand**: demandLogs, demandWeeklySummaries **Analytics**: auditLogs, restockSuggestions, branchScores **Promotions**: promotions **Reservations**: reservations **Storefront**: customers, customerAddresses, carts, cartItems, orders, orderItems, shipments **Engagement**: wishlists, reviews, vouchers, voucherRedemptions, loyaltyAccounts, loyaltyTransactions, notifications, recentlyViewed, sizeCharts, banners

11.3 User Roles (8)

`admin` · `manager` · `cashier` · `hqStaff` · `warehouseStaff` · `viewer` · `driver` · `supplier`

11.4 Route Groups (7)

`(auth)` · `(customer)` · `/admin` · `/branch` · `/pos` · `/warehouse` · `/driver` · `/supplier`